DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-1

ASSIGNMENT 1 – CO1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Text Normalisation and Search for a Hospital Record System

A multi-speciality hospital holds ten years of discharge summaries, prescriptions, operation notes and nursing observations as free text. Clinicians searching those records miss relevant cases constantly: a search for fracture does not return fractures, fractured or fracturing; nephrectomy is never connected to nephrectomies or to renal; patient identifiers, drug strengths and dates appear in a dozen different formats; and the same drug may be written in full, abbreviated or misspelt. No meaning can be extracted and no statistical model can be trained until this text is tokenised, normalised and reduced to canonical forms.

You are the language-engineering team. Before any machine learning is attempted, the hospital has asked you to build the finite-state layer: the regular expressions that recognise structured entities, the automata that validate them, the morphological analyser that relates every surface word to its stem and features, and the stemmer that collapses search terms. You must investigate what each formalism can and cannot express, implement them, measure them, and justify the design you recommend.

Planning data and constraints

- Corpus: at least 500 de-identified clinical records (real or synthetic, but documented), totalling at least 25,000 word tokens and covering at least four record types.
- Pattern inventory: at least 12 structured entity types to be recognised by regular expression — patient identifier, admission and discharge date, drug name with

strength, dosage frequency, laboratory value with unit, blood pressure, temperature, ICD code, ward and bed, contact number, age, and duration of symptom.

- Automata: at least four finite-state automata, each with at least six states, including one entity format that cannot be recognised conveniently without non-determinism.
- Morphological resources (build and document your own): a lexicon of at least 300 stems, at least 20 inflectional affixes and at least 15 derivational affixes, with irregular forms listed separately.
- Evaluation data: at least 40 clinician queries with relevance judgements over the corpus, and at least 200 word forms hand-annotated with their stem and features.
- Any manual effort (pattern writing, lexicon building, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The system must now ingest notes captured by optical character recognition from scanned handwritten forms, and notes typed in Romanised regional language mixed with English. Re-evaluate your original patterns, automata and analyser and decide whether to retain, modify or redesign them.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis 3 Marks
Guiding Enquiry

Which levels of linguistic analysis does this problem actually touch, and which family of models and algorithms is appropriate at each level?

Investigate and interpret:

- Describe the levels of linguistic analysis — phonology, morphology, syntax, semantics, pragmatics and discourse — and state, with an example drawn from your own corpus, which levels the record-search problem requires and which it can safely ignore.
- Distinguish the three model families used throughout this unit: state machines, formal rule systems and probabilistic models. Name one task in the hospital problem that each family suits best and say why the other two would be the wrong choice for it.
- Explain why ambiguity is the central problem of natural language processing, and identify at least four distinct ambiguities present in your corpus — lexical, morphological, segmentation and abbreviation ambiguity.
- Identify where search appears in this unit (state-space search in automata, backtracking in matching) and state the resources each consumes: time, memory, annotated data and expert effort.
- Set out, as a labelled diagram, the pipeline you will build in the remaining tasks, naming the input, the output and the formalism of every stage.

Evidence Expected

A reasoned problem framing that maps the hospital task onto levels of analysis, model families and a concrete, staged pipeline.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis (3 Marks)

Original task prompt

Guiding Enquiry

Which levels of linguistic analysis does this problem actually touch, and which family of models?

and algorithms is appropriate at each level?

Investigate and interpret:

Answer

1.1 Levels of linguistic analysis

The working corpus is a documented synthetic sample representing discharge summaries, prescriptions, operation notes and nursing observations. The required levels are morphology (fracture/fractures/fractured), tokenization and light syntax (keeping “500 mg” together and

separating dosage instructions), and limited semantics through entity types and terminology mappings. Phonology can be ignored for clean typed English, but becomes relevant indirectly when OCR or Romanized speech-like spelling introduces sound-based variants. Pragmatics and discourse interpretation are outside the first release: the system should retrieve records, not infer a diagnosis from the whole clinical narrative.

1.2 Model families

State machines suit exact structured validation, such as a date or patient ID, because transitions and acceptance are explicit. Formal rule systems suit spelling and morphological rules such as plural formation, because they are interpretable and testable. Probabilistic models suit noisy OCR correction or language identification, where several candidates are plausible. A probabilistic model is unnecessary for a fixed ID format; a hand-written rule set is brittle for every OCR typo; and a finite automaton alone does not infer a likely correction.

1.3 Ambiguity

Lexical ambiguity: “mass” may mean a lesion or a quantity. Morphological ambiguity: “treated” may be analysed as treat + past tense or as an adjective in context. Segmentation ambiguity: “12/05/26” may be a date in different day/month conventions, and “500mg” may be split or kept together. Abbreviation ambiguity: “MS” may refer to multiple sclerosis, morphine sulfate or another local term. These are controlled using context, metadata and explicit terminology tables; uncertain cases should retain alternatives rather than silently guess.

1.4 Search and resources

Automaton recognition explores states and transitions; deterministic recognition is O(n) in input length and uses O(1) working state for a fixed machine. Backtracking regex engines can take much longer on poorly designed patterns, so patterns should avoid nested ambiguous quantifiers. Building a lexicon and annotating 200 forms require expert time; a statistical correction layer requires representative labelled examples and more compute. The assignment does not provide an actual corpus or annotations, so all numeric results in this report are labelled as illustrative pilot estimates, not measured hospital outcomes.

1.5 Pipeline

Raw records → (1) Unicode cleanup and record metadata [normalization rules] → (2) tokenization [regex + maximal munch] → (3) structured entity detection [regex validators / DFA] → (4) abbreviation, date and unit canonicalization [ordered rewrite rules] → (5) morphological analysis [lexicon + finite-state transducer] → (6) fallback Porter stemming for unknown English terms [NLTK] → (7) indexed canonical tokens and query expansion [search layer] → (8) relevance evaluation [precision, recall, F1]. Input is free-text clinical notes; output is searchable records with token spans, canonical values, morphology and provenance.

TASK 2 – Basic Regular Expression Patterns 4 Marks

Guiding Enquiry

What can the basic regular expression operators express, and how are they combined into patterns that match real clinical strings exactly?

Investigate and interpret:

- Define the basic operators — concatenation, disjunction, Kleene star and plus, optionality, character classes, ranges, negated classes, anchors and counters — and give one minimal clinical example of each.
- Write and document a regular expression for each of the twelve entity types in your pattern inventory, stating for every pattern both what it must match and what it must not.
- Demonstrate greedy and non-greedy matching on a clinical sentence containing two dosage expressions, and explain precisely why the two quantifiers return different spans.
- Explain operator precedence and grouping, and show with a worked example how a misplaced disjunction silently changes the language the expression denotes.
- Evaluate every pattern over the corpus, reporting true positives, false positives and false negatives, and name the specific linguistic cause of each error class.

Evidence Expected

A documented pattern inventory with precise match specifications and a per-pattern error analysis carried out on real data.

2.1 Regular Expression Operators

Regular expressions (Regex) are patterns used to identify and extract specific information from clinical records. The following operators are useful for extracting patient details, medication information, laboratory results, and other clinical entities.

1. **Concatenation:** Combines pattern elements in sequence. For example, BP: followed by a value can identify blood pressure information.
2. **Disjunction (|):** Matches either of two alternatives. For example, mg|mcg matches either unit.
3. **Kleene Star (*):** Matches zero or more occurrences. For example, [0-9]* matches zero or more digits.

4. **Plus (+):** Matches one or more occurrences. For example, [0-9]+ matches one or more digits.
5. **Optionality (?):** Makes the preceding element optional. For example, (?:\s*mg)? permits an optional medicine unit.
6. **Character Class ([]):** Matches a character from a specified set. For example, [A-Z] matches one uppercase letter.
7. **Range:** [0-9] matches any single digit between 0 and 9.
8. **Negated Character Class:** [^\n]+ matches one or more characters other than a newline.
9. **Anchors (^ and $):** Specify the beginning and end of a line, respectively.
10. **Repetition Counter:** \d{4} matches exactly four digits, such as the year in a date.

These operators help identify structured clinical information from unstructured medical text.

2.2 Twelve Entity Extraction Patterns

The following Python regular expressions are designed to identify twelve types of information in clinical records. The patterns are case-insensitive where appropriate. They identify text that matches the specified format; additional validation is required to confirm that extracted information is clinically valid.

The patterns are initial extraction rules. For example, a date pattern may identify a date-shaped string without verifying whether the date exists in the calendar. Similarly, an ICD-like code match does not establish that the code is valid for a particular diagnosis. Clinical validation should therefore be performed after extraction.

2.2 Regular Expression Patterns for Entity Extraction

S. No.	Entity Type	Regular Expression Pattern	Example
1	Patient ID	\bP[0-9]{4,8}\b	P12345
2	Patient Age	\bAge\s*:\s*[0-9]{1,3}\b	Age: 42
3	Blood Pressure	\bBP\s*:\s*[0-9]{2,3}/[0-9]{2,3}\b	BP: 120/80
4	Heart Rate	\bHR\s*:\s*[0-9]{2,3}\b	HR: 85
5	Temperature	\bTemp\s*:\s*[0-9]{2}(?:\.[0-9])?\s*C\b	Temp: 38.2 C
6	Discharge Date	\bDischarge\s*:\s*\d{4}-\d{2}-\d{2}\b	Discharge: 2026-02-10
7	ICD-like Code	\b[A-TV-Z][0-9][A-Z0-9](?:\.[A-Z0-9]{1,4})?\b	A12
8	Medication Dosage	\b\d+(?:\.\d+)?\s*(?:mg|mcg|g|ml)\b	500 mg
9	Email Address	\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b	patient@example.com
10	Phone Number	\b\d{10}\b	9876543210
11	Oxygen Saturation	\bSpO2\s*:\s*\d{2,3}%?\b	SpO2: 98%
12	Glucose Level	\bGlucose\s*:\s*\d{2,3}\s*mg/dL\b	Glucose: 110 mg/dL

2.3 Python Implementation

The following program tests the twelve regular expressions against a sample clinical record. It prints the extracted information for each pattern.

```
import re
patterns = {
    "Patient ID": r"\b(?:MRN|PT)-?\d{6,10}\b",
    "Admission Date": r"\b(?:Admit(?:ted)?\s*:?\s*)?(\d{4}-\d{2}-\d{2})\b",
    "Discharge Date": r"\bDischarge(?:d)?\s*:?\s*(\d{4}-\d{2}-\d{2})\b",
    "Drug and Strength": r"\b[A-Za-z][A-Za-z-]*\s+\d+(?:\.\d+)?\s*(?:mg|mcg|g|mL|IU)\b",
    "Medication Frequency":
r"\b(?:once|twice|thrice)\s+daily\b|\b(?:OD|BD|TDS|QID|q\d+h)\b",
    "Laboratory Value":
r"\b(?:Hb|WBC|RBC|glucose|creatinine)\s*[:=]?\s*\d+(?:\.\d+)?\s*(?:g/dL|mg/dL|mmol/L|10\^9
/L)\b",
    "Blood Pressure": r"\b(?:BP\s*)?\d{2,3}/\d{2,3}\s*mmHg\b",
    "Temperature": r"\b(?:Temp\.?\s*)?\d{2}(?:\.\d+)?\s*°?\s*[CF]\b",
```

```
    "ICD-like Code": r"\b[A-TV-Z][0-9][A-Z0-9](?:\.[A-Z0-9]{1,4})?\b",
    "Ward and Bed": r"\bWard\s*[A-Z0-9-]+\s*[,;/ ]\s*Bed\s*[A-Z0-9-]+\b",
    "Contact Number": r"\b(?:\+?91[- ]?)?[6-9]\d{9}\b",
    "Patient Age": r"\b(?:Age\s*:?\s*)?(?:0|[1-9]\d?|1[01]\d|120)\s*(?:years?|y/o)\b"
}
record = """
Patient ID: MRN123456
Admit: 2026-02-03
Discharge: 2026-02-10
Medicine: Paracetamol 500 mg
Frequency: twice daily
Lab: glucose 6.5 mmol/L
BP: 120/80 mmHg
Temp: 38.2 C
Diagnosis code: A12
Location: Ward A, Bed 12
Contact: 9876543210
Age: 42 years
"""
print("REGULAR EXPRESSION EXTRACTION RESULTS")
print("-" * 45)
for entity, pattern in patterns.items():
    match = re.search(pattern, record, re.IGNORECASE)
    if match:
        print(entity + ":", match.group(0))
    else:
        print(entity + ": No match")
print("\nPositive Patient ID Test:",
      bool(re.search(patterns["Patient ID"], "MRN123456")))
print("Negative Patient ID Test:",
      bool(re.search(patterns["Patient ID"], "MRN12")))
```

2.4 Output

```
>>>
========= RESTART: C:/Users/Reddy/Documents/NLP ASSESSMENTS/ass 1 q1.py ========
============================================================
TASK 2: REGULAR EXPRESSION TEST RESULTS
============================================================
Patient ID: ['MRN123456']
Admission Date: ['2026-02-01']
Discharge Date: No match in this sample
Drug Strength: ['Paracetamol 500 mg']
Frequency: ['BD']
Lab Value: No match in this sample
Blood Pressure: ['BP 120/80 mmHg']
Temperature: No match in this sample
ICD Code: No match in this sample
Ward and Bed: No match in this sample
Contact Number: No match in this sample
Age: No match in this sample
Symptom Duration: No match in this sample

Positive patient ID test: True
Negative patient ID test: False
```

2.5 Evaluation Protocol

The patterns should be evaluated using a manually annotated set of clinical records. Each extracted entity should be compared with its correct annotated value. Precision, recall, and F1-score can be calculated as follows:

- **Precision** = True Positives / (True Positives + False Positives)
- **Recall** = True Positives / (True Positives + False Negatives)
- **F1-score** = 2 × Precision × Recall / (Precision + Recall)

A true positive is a correctly extracted entity, a false positive is an incorrect extraction, and a false negative is an entity that the system fails to identify. Evaluation should include different clinical note formats and variations in punctuation, spacing, abbreviations, and units.

The sample program demonstrates the extraction process but does not establish performance on a large clinical dataset. A reliable evaluation requires annotated clinical records and measured results.

TASK 3 – Regular Expressions for Tokenization and Normalization 4 Marks

Guiding Enquiry

How far can a purely regular layer take raw clinical text towards a clean, searchable token stream, and exactly where does it fail?

Investigate and interpret:

- Specify your tokenizer as a single regular expression, defining precisely what counts as a token — hyphenated terms, abbreviations containing full stops, decimal values, units written against numbers — and justify each decision.
- Apply maximal-munch (longest-match) tokenisation and show on at least three examples how the result differs from a naive split on whitespace.
- Build substitution rules that fold case, expand at least thirty clinical abbreviations and canonicalise date and unit formats; show the complete before-and-after text of one full record.
- Identify at least three normalisation decisions that are irreversible because information is destroyed, and argue when that loss is acceptable and when it is not.
- Give one requirement of the hospital task that no regular expression can express, and argue informally why — for example, matching balanced parentheses in a nested clinical note.

Evidence Expected

A working regular-expression tokenizer and normaliser with worked examples and a clear statement of the limits of regularity.

3. Tokenization and Text Normalization

3.1 Tokenizer

Tokenization is the process of dividing a sentence into smaller units called tokens. In medical text processing, a tokenizer should identify words, abbreviations, numbers, dates, measurements, and punctuation correctly.

A practical regular expression for tokenization is:

```
(?:[A-Za-z]+(?:\.[A-Za-
z]+)*|\d+(?:[.,]\d+)*(?:/\d+)?(?:\s?(?:mg|mcg|g|mL|mmHg|°C|°F|%))?|[A-Za-z]+(?:-[A-Za-
z]+)+|[^\w\s])
```

This expression helps identify words, abbreviations, numerical values, measurements, hyphenated words, and punctuation. For example, terms such as “Dr.”, “follow-up”, “3.5”, and “500mg” can be recognized according to the tokenizer rules. The original character positions should also be stored to maintain traceability to the source text.

Example:

Input: Patient has fever and takes 500mg medicine.

Output: Patient, has, fever, and, takes, 500mg, medicine, .

3.2 Maximal Munch

Maximal munch is a tokenization technique that selects the longest meaningful match available according to the defined rules. It helps prevent meaningful words, abbreviations, and measurements from being split unnecessarily.

Examples:

Input Text	**Ordinary Splitting**	**Preferred Tokenization**
BP 120/80 mmHg	BP, 120/80, mmHg	BP, 120/80 mmHg
Dr. Rao	Dr, ., Rao	Dr., Rao
follow-up	follow, -, up	follow-up
5.2 mg/dL	5.2, mg, /, dL	5.2 mg/dL

A suitable tokenizer should recognize decimal numbers, medical units, abbreviations, and hyphenated terms. The rules should be applied in the correct order so that longer and more specific patterns are recognized before general punctuation patterns.

3.3 Substitution Rules and Example

Substitution is the process of replacing medical abbreviations with their corresponding full forms. It improves readability and helps standardize medical text for further processing.

The following substitution rules can be used:

Abbreviation	Full Form
q.d. / OD	Once daily
b.i.d. / BD	Twice daily
t.i.d. / TDS	Three times daily
q.i.d. / QID	Four times daily
BP	Blood pressure
Temp	Temperature
SOB	Shortness of breath
N/V	Nausea/vomiting
DM	Diabetes mellitus
HTN	Hypertension
CAD	Coronary artery disease
CKD	Chronic kidney disease
UTI	Urinary tract infection
CXR	Chest X-ray
ECG	Electrocardiogram
CBC	Complete blood count
Hb	Haemoglobin
WBC	White blood cell count
RBC	Red blood cell count
Na	Sodium
K	Potassium
Cr	Creatinine
IV	Intravenous
IM	Intramuscular
PO	By mouth
PRN	As needed
NPO	Nothing by mouth
Rx	Prescription
FU	Follow-up

Example Record:

Admit: 01/02/2026. Pt c/o fever; BP 120/80 mmHg. Rx: Paracetamol 500mg BD. Discharge: 03/02/2026.

Normalized Output:

```
============================================================
TASK 3: TOKENISATION AND NORMALISATION
============================================================
Original text: Patient has SOB and BP 120/80 mmHg. Take medicine BD.
Tokens: ['Patient', 'has', 'SOB', 'and', 'BP', '120/80 mmHg', '.', 'Take', 'medi
cine', 'BD', '.']
Normalised text: Patient has shortness of breath and blood pressure 120/80 mmHg.
 Take medicine twice daily.

============================================================
```

Admit date: 2026-02-01; patient complains of fever; **blood pressure:** 120/80 mmHg; **prescription:** Paracetamol 500 mg twice daily; **discharge date:** 2026-02-03.

Result

Tokenization and text normalization help convert unstructured medical text into organized, standardized information. They improve readability, preserve important medical measurements, and support further Natural Language Processing tasks.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence 5 Mark

Guiding Enquiry

How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

Investigate and interpret:

- Define a finite automaton formally as the five-tuple (Q, Σ, δ, q_0, F), then design a deterministic automaton for at least four validators from your pattern inventory, giving both the transition diagram and the transition table for each.
- Design a non-deterministic automaton with ε-transitions for one entity whose format has optional parts, and trace its behaviour on one accepted and one rejected string, showing the set of states live at every step.
- Convert that NFA to a DFA by subset construction, showing the complete construction table, then minimise the DFA and report the state counts before and after.
- Apply Thompson's construction to turn one Task 2 regular expression into an automaton, and derive a regular expression back from one of your automata, demonstrating the equivalence in both directions.
- State the pumping lemma for regular languages and use it to prove that one specific clinical pattern is not regular — for example a note requiring equal numbers of opening and closing brackets — and relate the proof to your finding in Task 3.

Evidence Expected

Correct automata with full subset-construction and minimisation tables, a demonstrated two-way equivalence with regular expressions, and a valid non-regularity proof.

Answer

4. Finite Automata and Regular Languages

4.1 Formal Definition and Four DFAs

A Deterministic Finite Automaton (DFA) is a mathematical model used to recognize patterns in strings. It consists of five components:

DFA = (Q, Σ, δ, q_0, F)

- **Q:** A finite set of states.
- **Σ:** A finite input alphabet.
- **δ:** The transition function that determines the next state.

- **q_0:** The initial state.
- **F:** The set of accepting states.

The following four DFAs can be used to recognize common patterns.

(a) Six-Digit ID

This DFA accepts strings containing exactly six digits. It starts at state q_0 and moves to the next state for every digit. The string is accepted after the sixth digit.

Transition sequence: $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4 \rightarrow q_5 \rightarrow q_6$ (Accept)

Each transition consumes one digit. Any non-digit or additional digit leads to a dead state.

(b) ISO Date Format (YYYY-MM-DD)

This DFA recognizes strings having the format YYYY-MM-DD. It checks for four digits, a hyphen, two digits, another hyphen, and two final digits.

Example: 2026-02-10

The string is accepted only when the complete pattern is matched. Additional validation is required to check whether the date is a valid calendar date.

(c) Blood Pressure Format

This DFA recognizes blood pressure values containing two or three digits, followed by a slash and another two or three digits.

Pattern: D{2,3}/D{2,3}

Example: 120/80

The DFA checks the number of digits before and after the slash. Clinical validation is required to determine whether the blood pressure value is medically reasonable.

(d) Ward-Bed Label

This DFA recognizes labels containing the word Ward, a ward identifier, a separator, the word Bed, and a bed identifier.

Example: WardA-Bed12

The DFA moves through states as it reads each part of the label. It accepts the input only when all required parts are present.

All four DFAs use deterministic transitions. Invalid inputs can be directed to a dead state. The DFAs check the structure of the input, while additional validation checks its meaning.

4.2 Optional-Part NFA

A Nondeterministic Finite Automaton (NFA) can recognize a pattern in which a prefix is optional. Consider a pattern containing an optional MRN- prefix followed by six digits.

Examples of accepted strings:

- MRN-123456
- 123456

Example of a rejected string:

- MRN-12345

The NFA begins at the initial state and has two possible paths. One path consumes the prefix MRN-, while the other skips it using an epsilon (ε) transition. Both paths then continue through six-digit transitions.

- For the input 123456, the NFA follows the path that skips the optional prefix and reaches the accepting state after reading all six digits.
- For the input 12345, only five digits are read. The accepting state is not reached, so the string is rejected.
- Thus, an NFA can represent optional components using epsilon transitions and alternative paths.

4.3 Subset Construction and Minimization

Subset construction is a method used to convert an NFA into an equivalent DFA. In this process, each DFA state represents a set of NFA states.

The procedure is as follows:

1. Find the epsilon closure of the NFA's initial state.
2. For each input symbol, calculate the states reachable from the current set.

3. Find the epsilon closure of the resulting states.
4. Create a new DFA state for every previously unseen set.
5. Mark a DFA state as accepting if its set contains an accepting NFA state.
6. Repeat until no new reachable states are found.

- After constructing the DFA, minimization can be performed to reduce the number of states without changing the language recognized by the automaton.
- Equivalent states are identified and merged. A dead state is retained when necessary to represent invalid transitions in a complete DFA.

For the optional-prefix example, the exact transition table and state count depend on the concrete NFA definition and the input-symbol classes used. Therefore, the reachable subsets should be generated from the defined NFA before reporting the final table or state count.

4.4 Thompson Construction and Equivalence

Thompson's construction is a method for converting a regular expression into an NFA using transitions and epsilon edges. Consider the regular expression:

A(B|C)

The expression contains the symbol A, followed by either B or C.

The construction works as follows:

1. Create an initial transition for the symbol A.
2. Create an epsilon split to represent the alternatives B and C.
3. Construct one branch for B and another for C.
4. Join both branches using epsilon transitions.
5. Connect the joined branches to the final accepting state.

The resulting NFA accepts the strings AB and AC, which are exactly the strings described by the regular expression. Automata can also be converted into regular expressions through state elimination. During state elimination, the expression is updated using the rule:

Rij := Rij ∪ Rik(Rkk)*(Rkj)

This combines existing paths with paths that pass through an intermediate state, including repeated transitions on that state. Intermediate states are eliminated until the start and final states remain.

The equivalence of the regular expression and automaton can be tested by comparing their acceptance results for strings generated from the same alphabet.

4.5 Pumping Lemma

The pumping lemma is a property of regular languages that can be used to prove that certain languages are not regular. If a language L is regular, there exists a pumping length p such that every string s in L with length at least p can be divided into three parts:

s = xyz

These parts satisfy the following conditions:

1. $|xy| \leq p$
2. $|y| > 0$
3. $xy^iz \in L$ for every integer $i \geq 0$

Consider the language:

$L = \{ (^n)^n : n \geq 0 \}$

This language contains strings with equal numbers of opening and closing brackets, such as (), (()), and ((())).

Assume that L is regular and let p be its pumping length. Choose the string:

$s = (^p)^p$

Since $|xy| \leq p$, the substring y consists only of opening brackets because the first p characters are all opening brackets. If we pump y down by choosing $i = 0$, the resulting string contains fewer opening brackets than closing brackets. Therefore, the resulting string does not belong to L.

Result

Finite automata and regular-language techniques were studied through DFA construction, optional-prefix NFA design, subset construction, automata minimization, Thompson's construction, and the pumping lemma. These methods help identify valid patterns and explain the limitations of regular languages.

TASK 5 – Inflectional Morphology 3 Marks

Guiding Enquiry

Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

Investigate and interpret:

- Define morpheme, stem, affix, inflection and paradigm, and separate inflection from derivation using the standard criteria of category preservation, productivity and position relative to other affixes.
- List every inflectional paradigm present in your corpus — noun number, verb tense and aspect, subject agreement, adjective comparison — and give the full paradigm for at least five clinical lexemes.
- State the orthographic rules governing English inflection — e-deletion, consonant doubling, y-to-i, the sibilant -es rule — precisely enough to be implemented, and give for each one a counter-example on which it over-generates.
- Identify the irregular forms in your corpus and explain why each must be listed in the lexicon rather than derived by rule.
- Measure the effect of inflection: report the type-to-lemma ratio of your corpus and the proportion of failures in your query set that inflection alone explains.

Evidence Expected

Precisely stated inflectional rules with paradigms, an irregular list, and a quantified measurement of the effect of inflection on retrieval.

Answers: -

5. Morphology and Inflection

5.1 Definitions and Distinction

A morpheme is the smallest meaningful unit of a word. A stem is the base form to which an affix is attached. An affix is a bound morpheme added to a word or stem. Inflection changes the grammatical form of a word without creating a new lexeme. A paradigm is the set of grammatical forms associated with the same lexeme. Inflection usually preserves the word class and basic meaning of a word. For example, fracture and fractures refer to the same basic concept but differ in grammatical number. In contrast, derivation may change the meaning or grammatical category of a word. For example, operate is a verb, whereas operation is a noun.

Inflectional affixes are generally productive and often appear outside derivational affixes. For example, in nephrectomies, the derivational element forms a medical noun, while the plural ending indicates grammatical number.

5.2 Paradigms

A paradigm is a collection of related word forms that represent different grammatical forms of the same lexeme. Clinical language contains several types of paradigms.

A. Noun Number

Singular	Plural
Fracture	Fractures
Lesion	Lesions
Infection	Infections
Biopsy	Biopsies
Diagnosis	Diagnoses

B. Verb Tense and Aspect

Base Form	Third-Person Singular	Past Tense	Present Participle
Treat	Treats	Treated	Treating
Admit	Admits	Admitted	Admitting
Bleed	Bleeds	Bled	Bleeding
Heal	Heals	Healed	Healing
Prescribe	Prescribes	Prescribed	Prescribing

C. Adjective Comparison

Positive	Comparative	Superlative
Mild	Milder	Mildest
Severe	More severe	Most severe

These grammatical forms may appear in clinical records and medical reports. However, clinical notes often contain sentence fragments and may omit subjects or agreement markers.

5.3 Orthographic Rules

Orthographic rules describe spelling changes that occur when suffixes are added to words.

Rule	Explanation	Example	Exception or Counterexample
Adding -s	Add -s to most nouns to form the plural.	Fracture → Fractures	Diagnosis → Diagnoses, not diagnosiss.
Adding -es	Add -es to many nouns ending in s, x, z, ch, or sh.	Rash → Rashes	Virus → Viruses; spelling patterns are not always purely mechanical.
Final deletion	Remove the final *e* before adding -ing in many words.	Operate → Operating	Dye → Dyeing, to avoid ambiguity.
Consonant doubling	Double the final consonant in certain words before adding a suffix.	Admit → Admitted	Visit → Visited, not visitted.
Consonant + y	Change consonant + y to i before certain suffixes.	Biopsy → Biopsies	Play → Played; vowel + y remains unchanged.

These rules are useful for morphological analysis, but they require lexical exceptions. Stress patterns and spelling conventions must also be considered when determining whether a consonant should be doubled.

5.4 Irregular Forms and Retrieval Effect

Some words have irregular forms that cannot be generated reliably by applying ordinary suffix rules. Examples include:

Base Form	Irregular or Related Form
Bleed	Bled
Be	Was / Were
Have	Has / Had
Go	Went
Diagnosis	Diagnoses
Analysis	Analyses
Child	Children

These forms should be stored in an exception lexicon because they cannot all be derived using one productive suffix rule. A morphological analyzer can consult this lexicon when processing clinical documents.

Illustrative Calculation

Suppose a vocabulary contains 200 annotated word forms representing 140 distinct lemmas.

Type-to-lemma ratio = Total annotated forms ÷ Number of distinct lemmas

= 200 ÷ 140

= 1.43

This means that, on average, each lemma is represented by approximately 1.43 annotated forms. Suppose 12 out of 40 pilot query failures are caused solely by inflectional differences. Percentage of failures caused by inflection = (12 ÷ 40) × 100

= 30%

These values are illustrative only. They must be replaced with actual counts from the clinical corpus and the 40 evaluated queries before reporting experimental findings.

Result

Morphological analysis helps identify the internal structure of words and their grammatical variations. Inflectional paradigms, orthographic rules, and exception lexicons support the normalization of clinical text. This improves the identification of related word forms and can help reduce retrieval failures caused by inflectional differences.

TASK 6 – Derivational Morphology 4 Marks

Guiding Enquiry

How does derivation build the specialised clinical vocabulary, and why is it harder to handle than inflection?

Investigate and interpret:

- Define derivation and list at least fifteen derivational affixes attested in your corpus — for example -itis, -ectomy, -ology, -osis, -aemia, -ation, -able, -ive, nephro-, hyper-, hypo-, re-, un- — stating the category change each one causes.
- Build the derivational family of at least six clinical roots, showing the order in which affixes attach and identifying at least two orders that are impossible.
- Explain semantic non-compositionality with clinical examples in which the meaning of the derived word cannot be predicted from its parts, and state what this implies for a search system that stems aggressively.
- Compare the productivity of your affixes empirically: count types and tokens for each in the corpus and classify each affix as productive or fossilized, with the evidence for the classification.
- Decide whether derivational relatives should be conflated for search, citing at least two cases where conflation helps the clinician and two where it produces a clinically misleading match.

Evidence Expected

A documented derivational analysis with an affix inventory, productivity measurements and a reasoned position on conflation.

Answers: -

6. Derivational Morphology

6.1 Derivational Affixes

Derivational affixes are prefixes or suffixes added to a root or stem to create a new word or change its meaning or grammatical category. In medical terminology, derivational affixes help form words related to diseases, treatments, body parts, and medical professions.

Prefixes often preserve the grammatical category of a word while changing its meaning. Suffixes frequently change the grammatical category, although their effects depend on the word and the affix involved

Affix	Typical Meaning or Function	Example
-itis	Inflammation; noun	Tonsillitis
-ectomy	Surgical removal; noun	Nephrectomy
-ology	Field or study; noun	Neurology
-osis	Condition; noun	Fibrosis
-aemia	Blood condition; noun	Anaemia
-ation	Verb to event or process noun	Inflammation
-ment	Verb to noun	Treatment
-able	Verb to adjective	Treatable
-ive	Forms adjectives	Responsive
-al	Forms adjectives	Renal
-ic	Forms adjectives	Cardiac
-ist	Person associated with a field; noun	Specialist
hyper-	Excessive or above normal	Hypertension
hypo-	Low or below normal	Hypotension
nephro-	Kidney	Nephrology
cardio-	Heart	Cardiology
re-	Again	Re-treatment

6.2 Derivational Families and Order

A derivational family contains words formed from the same root or related base. These words may have different meanings or grammatical categories.

Root or Base	Related Forms
Operate	Operate → Operation → Operational
Infect	Infect → Infection → Infectious
Treat	Treat → Treatment → Treatable
Nephro	Nephrology → Nephrologist
Cardio	Cardio- + -logy → Cardiology
Fracture	Fracture → Fractured → Fracturing

The forms fractured and fracturing are primarily inflectional or participial forms rather than purely derivational forms.

The order of affixes is important because English follows particular word-formation patterns. For example, un + treatable produces untreatable, which is a valid English word. However, treat + able + un does not follow the standard formation pattern.

Therefore, a morphological analyzer must consider the order and function of affixes when identifying the structure of medical words.

6.3 Non-Compositionality and Stemming Risk

Non-compositionality occurs when the meaning of a word or phrase cannot be determined simply by combining the ordinary meanings of its individual parts. For example, heart attack refers to a medical condition rather than a physical attack on the heart. Renal failure refers to a clinical condition involving kidney function. The word cold may refer to a low temperature or an illness, depending on the context.

Aggressive stemming can create problems in medical information retrieval. It may merge medically distinct words or fail to recognize multiword clinical expressions. Removing suffixes alone is therefore insufficient for identifying clinical equivalence. A better approach combines morphological processing with curated synonyms, medical terminology resources, and standardized clinical codes. This helps distinguish words that look similar but have different medical meanings.

6.4 Productivity and Conflation

Morphological productivity refers to how commonly an affix is used to form new words. A corpus audit can measure productivity by counting the distinct word types and total token occurrences associated with each affix. For an illustrative classification, -ation, -al, -ic, un-, and re- can be considered relatively productive. The suffixes -ectomy, -itis, -aemia, and -osis are productive within medical terminology but are restricted by their specialized meanings. Some word endings, such as -th in growth, are fossilized forms and should not be treated as reliable productive rules.

Result: -Derivational morphology helps explain how medical words are formed from roots, prefixes, and suffixes. Understanding derivational families, affix order, non-compositional meanings, and morphological productivity improves clinical text normalization. Combining morphological analysis with medical terminology resources can reduce incorrect conflation and improve information retrieval.

TASK 7 – Finite-State Morphological Parsing 5 Marks

Guiding Enquiry

How can a transducer relate a surface form to its stem and features, and run in both directions on the same machine?

Investigate and interpret:

- Define a finite-state transducer as a two-tape automaton and explain the lexical and surface tapes with one fully worked clinical example.
- Build your lexicon as an FST organised into continuation classes, covering at least three parts of speech together with the inflectional and derivational affixes of Tasks 5 and 6.
- Hand-trace the parsing of at least five surface forms, showing the state sequence, the symbols consumed and emitted on each tape, and the final analysis — for example fractures → fracture +N +PL.
- Run the same transducer in the generation direction, producing surface forms from stem-plus-feature input, and show explicitly that parsing and generation use one machine and not two.
- Handle ambiguity: give at least two surface forms with more than one analysis, show the transducer returning all of them, and state how a downstream component would choose between them.

Evidence Expected

A working transducer specification with traced analyses in both directions and an explicit treatment of ambiguity.

Answers: -

7. Finite-State Transducers

7.1 Finite-State Transducer

A finite-state transducer (FST) is a finite-state machine that reads symbols from an input and produces corresponding output symbols. An FST represents the relationship between a lexical form and its surface form. The lexical form contains the lemma and grammatical tags, while the surface form represents the actual written word.

Example:

Lexical form: fracture+N+PL

Surface form: fractures

During morphological analysis, the FST reads the surface form and identifies its lexical representation. During generation, it reads the lexical representation and produces the corresponding surface word. Both operations use the same lexical-to-surface relationship in opposite directions.

7.2 Continuation Classes

Continuation classes organize word stems and affixes according to their grammatical properties. They determine which transitions are permitted after a particular stem or affix. The main continuation classes used in this system are NOUN, VERB, ADJECTIVE, and DERIVATION.

Noun stems can connect to plural rules, while verb stems can connect to third-person singular, past tense, and progressive forms. Adjective stems can connect to comparative and superlative endings. Derivational transitions connect stems to noun-forming, adjective-forming, and medical suffixes. For a complete resource, the lexicon should contain at least 300 stems, 20 inflectional affixes, and 15 derivational affixes. The examples in this report demonstrate the basic structure; they do not represent a complete 300-stem lexicon.

7.3 Five Hand-Traced Analyses

Morphological analysis identifies the stem and grammatical information represented by a surface word. The following examples illustrate the process.

Surface Form	Lexical Analysis	Explanation
fractures	fracture + N + PL	The noun fracture takes the plural ending -s.
fractured	fracture + V + PAST / fracture + ADJ + PART	The word may represent a past-tense verb or a participial adjective.
treated	treat + V + PAST / treat + ADJ + PART	The word may represent a past-tense verb or a participial adjective.
biopsies	biopsy + N + PL	The final y changes i before plural ending -es.
infections	infection + N + PL	The noun infection takes the plural ending -s.

Each analysis begins with a stem lookup and continues through the appropriate suffix transition. Spelling changes, such as the change from y to ies in biopsies, must be handled by

orthographic rules rather than simple concatenation. These examples show how an FST can associate a surface word with its lemma and grammatical features.

7.4 Generation and Ambiguity

Morphological generation converts a lexical representation into its corresponding surface form.

Examples:

Lexical Input	Generated Surface Form
fracture + N + PL	fractures
biopsy + N + PL	biopsies
treat + V + PAST	treated

Parsing and generation are inverse operations that use the same lexical-to-surface relationship. Parsing moves from a surface form to its possible lexical analyses, while generation moves from a lexical representation to the corresponding written form.

```
============================================================
TASK 7: MORPHOLOGICAL ANALYSIS
============================================================
fractures -> [('fracture', 'NOUN', 'PLURAL')]
biopsies -> [('biopsy', 'NOUN', 'PLURAL')]
infections -> [('infection', 'NOUN', 'PLURAL')]
treated -> [('treat', 'VERB', 'PAST'), ('treat', 'ADJECTIVE', 'PARTICIPLE')]
fractured -> [('fracture', 'VERB', 'PAST'), ('fracture', 'ADJECTIVE', 'PARTICIPL
E')]
hospital -> No analysis in the sample lexicon
```

The system should initially return all valid analyses rather than immediately selecting only one. A downstream contextual analyzer or clinical tagger can then rank the alternatives using information from the surrounding text. If the available evidence is insufficient to determine the correct interpretation, both analyses should be preserved.

Result

Finite-state transducers provide a systematic method for analyzing and generating morphological forms. Continuation classes organize stems and affixes, while hand-traced analyses demonstrate how words are decomposed into lexical forms and grammatical tags. Generation and ambiguity handling allow the system to produce valid word forms and preserve alternative interpretations when necessary.

TASK 8 – Orthographic Rules and Two-Level Morphology 4 Marks

Guiding Enquiry

How are spelling changes at morpheme boundaries handled without listing every surface form in the lexicon?

Investigate and interpret:

- Explain why a lexicon transducer alone cannot produce correct surface spellings, using at least three examples in which plain concatenation yields the wrong string.
- Write at least five orthographic rules in the standard rewrite notation (a → b / c ___d), stating the context of each rule precisely.
- Implement each rule as a transducer and compose the rule transducers with the lexicon FST; draw the cascade and state the order dependencies between the rules.
- Contrast this sequential cascade with the parallel two-level (Koskenniemi) formulation, and explain the practical advantage each has over the other.
- Test the complete system on at least twenty forms chosen to exercise every rule, tabulate successes and failures, and account for every failure.

Evidence Expected

Correct rule transducers, a documented composition order, and a tested cascade with full error accounting.

Answers: -

8. Orthographic Rules and FST Composition

8.1 Why Plain Concatenation Fails

Plain concatenation combines a word stem and a suffix without making the spelling changes required by English. This can produce incorrect word forms. For example, combining biopsy with the plural suffix -s produces biopsy+s instead of biopsies. Similarly, admit+ed produces admitted only when the final consonant is doubled, and operate+ing produces operating after the final e is removed. Therefore, a lexical finite-state transducer (FST) must be combined with orthographic rule transducers that handle spelling changes at morpheme boundaries.

8.2 Rewrite Rules

Rewrite rules describe the spelling changes required when suffixes are added to word stems.

S. No.	Rewrite Rule	Example
1	Change y to i before -es when y follows a consonant.	biopsy + es → biopsies
2	Delete final e before -ing in applicable words.	operate + ing → operating
3	Double the final consonant in eligible stressed consonant-vowel-consonant forms before -ed.	admit + ed → admitted
4	Add -es after sibilant endings such as s, x, ch, and sh.	rash + es → rashes
5	Change y to i before -ed when y follows a consonant.	copy + ed → copied

These rules should be applied at morpheme boundaries. Consonant doubling requires appropriate stress and spelling conditions, so lexical exceptions may be necessary. The rules must be designed to avoid incorrect transformations.

8.3 Cascade and Two-Level Comparison

An FST cascade applies multiple transducers in sequence to convert a lexical representation into a surface word.

The processing sequence is:

- Lexical stem + grammatical tags → Lexicon and continuation-class FST → Orthographic rule transducers → Surface form
- The orthographic stage handles changes such as y-to-i conversion, final-e deletion, consonant doubling, and suffix realization.
- The order of rules is important because one spelling transformation may affect the input to another rule. Each stage should therefore be tested independently to avoid unintended changes.
- A sequential cascade is easier to understand, debug, and maintain because each transducer performs a specific task.

A two-level morphological model represents relationships between lexical and surface symbols simultaneously through constraints. It can reduce some rule-order interactions, but its implementation may be more difficult for beginners to inspect and understand. Both approaches can support morphological generation, provided that their rules correctly represent the required spelling patterns.

8.4 Test Plan

The implementation should be tested using regular and irregular spelling patterns. The following table provides 20 test cases with their expected outputs. For each test case, the generated output should be compared with the expected output. The result should be recorded as Pass if both forms match or fail if they differ.

S. No.	Input Form	Expected Output
1	fracture + s	fractures
2	lesion + s	lesions
3	biopsy + es	biopsies
4	diagnosis + es	diagnoses

5	rash + es	rashes
6	box + es	boxes
7	operate + ing	Operating

The test report should contain the input form, expected output, generated output, and pass/fail status. A reliable accuracy score can only be reported after running the composed implementation and checking any exceptions. The forms listed above are expected outputs and are not claimed experimental results.

Result

Orthographic rewrite rules help finite-state transducers generate correctly spelled words from lexical stems and grammatical suffixes. Combining a lexicon transducer with spelling-rule transducers improves morphological generation. Systematic testing helps identify errors and verify whether the implementation produces the expected surface forms.

TASK 9 – The Porter Stemmer 5 Marks

Guiding Enquiry

What does a rule-based stemmer achieve with no lexicon at all, and what does it cost in precision?

Investigate and interpret:

- State the algorithm precisely: the definition of consonant and vowel, the measure m of a word, and the conditions *v*, *d and *o, with the calculation worked through for at least six clinical words.
- Work through all five steps of the algorithm by hand for at least six words, showing which rule fires at each step and the intermediate form after each one.
- Contrast stemming with lemmatisation using your Task 7 transducer: stem and lemmatise the same 200 annotated forms and tabulate where the two agree and where they diverge.
- Quantify both error types with clinical examples — over-stemming, which conflates words that must stay distinct, and under-stemming, which fails to conflate related words — and report the rate of each on your annotated set.
- Measure the retrieval effect: run your 40 queries three times, with no stemming, with the Porter stemmer and with full morphological analysis, and report precision, recall and F1 for each condition.

Evidence Expected

Correct hand-traces of the algorithm, a quantified error analysis, and a retrieval experiment comparing stemming against full analysis.

Answers: -

9. Porter Stemming and Lemmatization

9.1 Definitions and Conditions

The Porter stemmer is a rule-based stemming algorithm that removes or modifies suffixes to produce a word stem. It uses consonant and vowel patterns and a measure called m to determine whether certain suffix-removal rules can be applied. The measure m represents the number of vowel-consonant sequences in the relevant stem region.

Condition	Meaning	Condition
*v	The stem contains a vowel.	*v
*d	The stem ends with a double consonant.	*d

The Porter stemmer applies suffix rules in an ordered sequence rather than consulting a dictionary to find the correct lemma. The resulting stem may not be a valid English word.

9.2 Six Illustrative Traces

The following examples illustrate how stemming may process clinical and general English words. Exact intermediate forms should be obtained by running the selected Porter implementation because the applied rules depend on the algorithm's conditions and implementation.

S. No.	Input Word	Illustrative Processing	Expected Interpretation
1	fractures	Removal of the plural ending may produce fracture, followed by further suffix processing if applicable.	Stemming does not necessarily preserve the dictionary form.
2	infections	The plural ending is removed first. Later rules may process additional suffixes when their conditions are satisfied.	The final form depends on the applicable suffix rules and measure.
3	treated	The -ed ending may be removed if the stem satisfies the required conditions.	The resulting stem may be treat.
4	biopsies	The Porter algorithm may transform the plural form into a stem such as biopi.	The stem may differ from the dictionary lemma biopsy.
5	relational	Ordered suffix transformations may reduce the word to a shorter stem, such as relat.	The output depends on the Porter rules applied.
6	nephrectomies	General suffix-removal rules may fail to identify the intended medical lemma.	Specialized medical morphology may require additional rules.

These examples demonstrate that stemming is not the same as lemmatization. For accurate experimental reporting, the exact output of each word should be generated using the selected Porter stemmer and recorded along with its intermediate transformations.

9.3 Stemming vs. Lemmatization

Stemming and lemmatization are two methods used to normalize words in Natural Language Processing. Stemming removes or modifies word endings using rules. It does not necessarily produce a valid dictionary word. Lemmatization uses lexical information and grammatical features to identify the base form, known as the lemma.

9.4 Errors and Information Retrieval

Stemming can introduce two major types of errors: over-stemming and under-stemming. Over-stemming occurs when unrelated words are reduced to the same stem. For example, some stemming algorithms may incorrectly merge words such as university and universe. This can reduce retrieval precision by returning irrelevant documents.

Under-stemming occurs when related words remain separated because the algorithm does not recognize their morphological relationship. This can reduce retrieval recall, particularly for specialized medical terms that are not handled by general stemming rules.

To evaluate information retrieval, 40 clinician queries should be tested using three approaches:

1. No stemming.
2. Porter stemming.
3. Finite-state transducer (FST) analysis.

The following evaluation metrics should be calculated:

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1-score = 2 × Precision × Recall / (Precision + Recall)

Here, TP represents true positives, FP represents false positives, and FN represents false negatives.

An illustrative expectation is that a system without stemming may achieve higher precision but lower recall when word forms differ. Porter stemming may improve recall but introduce false positives. A curated FST may provide a better balance for clinical terms included in its lexicon, although it may fail to analyze unknown words and OCR errors.

These are expected trends, not measured findings. Actual precision, recall, and F1-scores require the 40 queries, relevance judgments, retrieval results, and the completed implementations.

```
============================================================
TASK 9: PORTER STEMMING
============================================================
fractures -> fractur
infections -> infect
connected -> connect
playing -> play
studies -> studi
biopsies -> biopsi
```

Result

Porter stemming provides a rule-based method for reducing words to stems, while lemmatization identifies their dictionary base forms. Comparing stemming, lemmatization, and finite-state morphological analysis helps identify normalization errors and understand their effects on clinical information retrieval. Reliable performance comparisons require actual annotated data and evaluated queries.

TASK 10 – Implement and Test the Pipeline 3 Marks

Guiding Enquiry

Can a working implementation reproduce the behaviour you derived by hand?

Investigate and interpret:

- Implement in Python the tokenizer, the entity patterns, the automata including the subset construction, the morphological transducer and the Porter stemmer, using the standard library and NLTK only.
- Run the complete pipeline over the corpus and report, for every stage, the number of items processed, matched or analysed, the number rejected, and the run-time.
- Include test cases for every automaton, at least one string that must be rejected per pattern, the ambiguous forms from Task 7 and the irregulars from Task 5, and check the output against your hand-computed results from Tasks 4, 7 and 9.
- Present the results in comparable tables and at least one plot — for example analysis coverage against lexicon size, or run-time against record length.

Evidence Expected

Working, documented code, a reproducible test procedure, and results tables and graphs.

Answers: -

10. Implement and Test the Pipeline

10.1 Implementation

The complete Natural Language Processing (NLP) pipeline is implemented in Python using regular expressions, finite automata, morphological analysis, and the Porter stemming algorithm. The pipeline processes clinical text through multiple stages and records the results produced at each stage.

The implementation includes the following components:

1. **Tokenization:** divides clinical text into tokens such as words, numbers, measurements, and punctuation.
2. **Entity extraction:** identifies patient IDs, ages, blood pressure, heart rate, temperatures, dates, medication dosages, and other defined entities.
3. **Finite automata:** checks input patterns such as six-digit IDs, ISO date formats, blood pressure values, and ward-bed labels.

4. **Subset construction:** demonstrates the conversion of NFA states into sets of states for deterministic processing.
5. **Morphological analysis:** identifies word stems and grammatical features, including plural forms and verb forms.
6. **Porter stemming:** applies the Porter stemming algorithm to reduce words to their stems.

The implementation also includes test cases for valid and invalid inputs, irregular word forms, and ambiguous morphological forms.

10.2 Test Procedure

The following procedure is used to test the implementation:

1. Prepare the clinical text corpus in a text or CSV file.
2. Execute the Python program using the selected corpus.
3. Tokenize each record and extract the entities using the defined patterns.
4. Test the finite automata using valid and invalid strings.
5. Perform morphological analysis and Porter stemming on the extracted words.
6. Compare the generated outputs with the expected results from Tasks 4, 7, and 9.
7. Record the number of records, tokens, entity matches, analyzed words, and execution time.
8. Generate result tables and a plot to compare processing time with record length.

The program saves record-level measurements, test results, and subset-construction information in separate output files.

10.3 Test Cases and Results

The following table describes the test cases included in the implementation.

S. No.	Component	Test Input	Expected Result
1	Six-digit ID	123456	Accepted
2	Six-digit ID	12345	Rejected
3	ISO date format	2026-02-10	Accepted
4	ISO date format	2026/02/10	Rejected
5	Blood pressure	120/80	Accepted
6	Blood pressure	8/80	Rejected

7	Ward-bed label	WardA-Bed12	Accepted
8	Ward-bed label	Ward-Bed	Rejected
9	Optional MRN prefix	MRN-123456	Accepted
10	Optional MRN prefix	MRN-12345	Rejected
11	Morphological analysis	fractures	Expected analysis: fracture + N + PL
12	Morphological analysis	biopsies	Expected analysis: biopsy + N + PL
13	Morphological analysis	treated	Expected analysis includes treat + V + PAST
14	Irregular morphology	bled	Expected analysis: bleed + V + PAST
15	Irregular morphology	children	Expected analysis: child + N + PL

The actual output of each test must be compared with the expected result. A test passes when the generated output matches the expected result.

```
============================================================
TASK 10: PIPELINE EXECUTION SUMMARY
============================================================

Record 1 : Admit: 2026-02-01. Patient has fractures. Paracetamol 500 mg BD.
Token count: 16
Entity matches: 3
Morphological analyses: 1

Record 2 : BP 120/80 mmHg; Temp 38.2 C; glucose 6.5 mmol/L.
Token count: 13
Entity matches: 3
Morphological analyses: 0

Record 3 : MRN123456, Age 42 years, symptoms for 3 days; Ward A, Bed 12.
Token count: 18
Entity matches: 4
Morphological analyses: 0

FINAL SUMMARY
Records processed: 3
Total tokens: 47
Total entity matches: 10
Tokens with sample morphological analyses: 1
Execution time (seconds): 0.181698

Testing completed successfully.
>>
```

10.4 Pipeline Performance

The implementation records the number of items processed and the execution time for each stage. The following table should be completed using the actual program output.

Stage	Items Processed	Matches or Analyses	Rejected or Unanalysed	Runtime (seconds)
Tokenization	Obtain from output	Tokens generated	Not applicable	Obtain from output
Entity extraction	Records processed	Entity matches	Records or patterns with no match	Obtain from output
Morphological analysis	Alphabetic tokens	Words analysed	Words not analysed	Obtain from output
Porter stemming	Alphabetic tokens	Words stemmed	Not applicable	Obtain from output

The number of rejected items should be calculated using a clearly defined rejection rule. A record with no entity match is not necessarily an invalid record.

10.5 Performance Visualization

A scatter plot of pipeline runtime against record length is generated to examine how processing time varies with the length of the clinical text.

The horizontal axis represents record length in characters, and the vertical axis represents pipeline runtime in seconds. Each point represents one processed record.

The graph can help identify whether longer records generally require more processing time. Actual conclusions should be drawn only after running the implementation on the selected corpus.

Result

The NLP pipeline combines tokenization, entity extraction, finite automata, subset construction, morphological analysis, and Porter stemming. Automated test cases compare generated outputs with expected results, while performance measurements and a runtime plot support evaluation of the implementation. Final performance conclusions require execution on the actual corpus.

TASK 11 – Compare, Recommend and Defend the Normalisation Design 2 Marks

Guiding Enquiry

Which normalisation design would you deploy in the hospital, and what evidence supports your choice?

Investigate and interpret:

- Compare the three conflation strategies — no normalisation, Porter stemming and full finite-state morphological analysis — on retrieval quality, coverage, run-time and maintenance effort, using your own measured results.
- Recommend a design, stating its expected accuracy, its failure modes, and what a clinician would actually notice when it fails.
- Discuss the manual effort involved in pattern writing, lexicon and affix construction and annotation, and identify the point at which a statistical or learned approach becomes more economical than hand-built finite-state resources.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the highest recall alone is not sufficient.

Answer

11.1 Comparison

No normalization is simplest and fast but misses inflectional variants. Porter stemming improves recall for common English suffixes with no lexicon maintenance, but may conflate clinically distinct words and mishandle medical forms. Finite-state analysis is interpretable, returns grammatical features and supports generation, but requires lexicon/affix maintenance and annotation. For a hospital, use layered normalization: preserve original text, canonicalize well-defined entities, apply a curated FST for known clinical terms, and use Porter only as a low-confidence fallback for unknown English tokens.

11.2 Recommendation and economics

Deploy the layered design behind a reviewable index, with hard validation for identifiers and units, explicit date locale, terminology mappings and a monitored fallback. Expected failure modes are OCR substitutions, ambiguous abbreviations, non-English morphology, unseen drugs and over-generalized derivation. Clinicians notice missing records or irrelevant matches, so show why a result matched and allow exact-search fallback. Manual resource construction is worthwhile for high-value stable vocabulary; a learned or weighted correction model becomes more economical when labelled noisy examples grow enough to cover many interacting spelling patterns. Never let a statistical correction silently alter the clinical source record.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The system must now ingest two new sources. The first is optical character recognition output from scanned handwritten forms, in which characters are routinely confused (0 for O, 1 for l, rn for m), words are broken across line endings, and diacritics are lost. The second is notes typed by staff in Romanised regional language mixed with English, for example “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. At least 20% of incoming tokens are expected to be out of lexicon, misspelt, or not English at all.

Re-evaluate your original design under the changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the properties of the new input that your patterns, automata and analyser cannot handle: character-level noise, words hyphenated across lines, out-of-lexicon stems, non-English morphology and token-level code-mixing.
- Identify the assumptions this breaks: clean typed input, a closed vocabulary, English-only morphology, and exact matching by automata that accept or reject with nothing in between.
- Estimate how the noise rate will affect pattern precision and analyser coverage, reasoning from the error rates you measured in Tasks 2 and 7.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

12A. Impact Analysis

The introduction of noisy OCR-generated clinical text and code-mixed language changes the assumptions on which the original NLP pipeline was designed. OCR may introduce character-level errors such as 0/O, 1/l, and rn/m confusion. It may also split words across lines, insert incorrect spaces, or remove diacritics. In addition, Romanised regional-language expressions introduce unfamiliar words, non-English morphology, and token-level code-mixing.

These changes violate four major assumptions of the original design: clean typed input, a closed vocabulary, English-only morphology, and exact matching by automata that either accept or reject an input. As a result, structured patterns may miss valid information, and the morphological analyser may fail to process unfamiliar or mixed-language words.

If 20% of the input tokens are noisy or outside the lexicon, pattern matching and morphological coverage may decrease considerably. However, the actual reduction depends on the type of noise, the affected tokens, and the robustness of each pattern. Exact numerical degradation cannot be established without the measured error rates from Tasks 2 and 7, a representative test sample, and verified reference labels.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended pipeline actually behave on the new input?

- Run or hand-trace at least three new records — one from OCR, one code-mixed, one both — through your original pipeline and record precisely what fails first and why.
- Measure the degradation: report pattern precision and recall, analyser coverage and retrieval F1 on the noisy sample against your clean-corpus baseline.
- Identify whether a different formalism — weighted automata, edit-distance matching, or a character n-gram language identifier — would handle the new input more economically than extending what you have.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

12B. Evaluation of the Original Design

Record A – OCR Noise

Input: MRN: PT-12O456; Temp 38.2 C

The character O is used instead of the digit 0 in the patient identifier. Therefore, the strict patient-ID pattern may reject the identifier, although the temperature pattern may still recognise 38.2 C.

Record B – Code-Mixed Input

Input: patient ku fever irundhuchu, paracetamol 500mg kuduthen. The pipeline may recognise the English clinical term fever and the drug strength 500mg. However, an English-only morphological analyser is unlikely to analyse the Romanised regional-language forms ku,

irundhuchu, and kuduthen correctly without suitable language resources. This demonstrates the limitations of a closed English vocabulary and English-only morphology.

Record C – Combined OCR Noise and Code-Mixing

Input: Disch arge: 03/02/2026; paracetamol 5O0mg BD

The word Disch arge is incorrectly split, and the drug strength contains the letter O instead of the digit 0. Consequently, the original pipeline may fail to recognise the discharge label or drug strength. The date may match a date-format pattern, but its interpretation remains dependent on the date convention used. This example demonstrates how multiple errors can affect different stages of the pipeline.

These cases show that the original design performs well with clean, expected input but has limited tolerance for OCR errors, unfamiliar vocabulary, and mixed-language text.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the finite-state layer be extended, replaced or supplemented?

- Choose retain, modify or redesign — for example placing a weighted edit-distance transducer in front of the analyser, extending the lexicon with regional stems, adding token-level language identification, or falling back to stemming for out-of-lexicon tokens.
- Describe the revised design and state exactly what changes in the patterns, the automata, the lexicon and the rule cascade.
- Determine the effect on coverage, precision, run-time and the annotation effort needed to maintain the system.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

12C. Adaptation and Redesign

Decision: Modify the existing design rather than replace it.

The finite-state layer remains useful for validating structured clinical information. However, additional processing stages are required to handle noisy and multilingual input.

1. **OCR Noise Handling:** Introduce conservative text normalisation to repair obvious spacing errors and line-end hyphenation. Character substitutions such as O and 0 should generate correction candidates rather than trigger unconditional replacement.
2. **Approximate Matching:** Use weighted edit distance to identify possible matches for noisy narrative terms. Candidate matches should be accepted only when their confidence is sufficient; uncertain cases should be sent for review.
3. **Token-Level Language Identification:** Use character n-gram features to estimate the likely language of individual tokens. This can help identify English words, regional-language forms, and code-mixed expressions.
4. **Extended Lexicon:** Add a small, reviewed regional-language lexicon containing relevant words, transliteration variants, and common morphological forms. This will improve the analysis of Romanised regional-language text.
5. **Morphological Fallback:** Retain the existing morphological analyser and use the Porter stemmer for likely English words that are not covered by the main lexicon. The Porter stemmer should not be applied indiscriminately to regional-language words.
6. **Strict Structured-Data Validation:** Preserve exact validation rules for patient identifiers, dates, medication strengths, and clinical measurements. When a possible correction is detected, generate a candidate and mark it for validation instead of silently accepting a fuzzy match.
7. **Traceability and Testing:** Store the original input, proposed correction, confidence score, and correction source. Extend the test suite with OCR errors, hyphenated words, out-of-lexicon stems, non-English morphology, and code-mixed tokens.

These adaptations are expected to improve coverage of noisy text. However, permissive matching may reduce precision, while additional processing can increase execution time and annotation effort.

D.Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one, given the evidence you generated?

- Use your pattern error analysis, automaton traces, coverage figures, retrieval measurements and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what a finite-state layer will never handle, however much it is extended.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

12D. Evidence-Based Defence

The evaluation indicates that the original finite-state and rule-based components should be retained for predictable formats and known morphological patterns. Their main limitation is that exact matching cannot reliably handle every OCR substitution, unfamiliar word, or code-mixed expression. A hybrid architecture is therefore recommended. Deterministic rules and automata should continue to validate structured clinical entities, while approximate matching and language identification should generate candidates for uncertain narrative text. Confidence scoring and human review should be used whenever the system cannot safely determine the intended value.

The revised pipeline should be evaluated using paired clean and noisy samples. The evaluation should measure pattern precision and recall, morphological analyser coverage, retrieval F1-score, execution time, and clinician review time. The results should be compared with the baseline measurements from Tasks 2 and 7. Actual numerical improvements must be reported only after these experiments have been performed. The adapted system will still have limitations. Finite-state methods cannot independently understand unrestricted clinical meaning, resolve every abbreviation from context, or reliably infer a diagnosis. Therefore, human review and a controlled clinical terminology resource remain necessary for safe and reliable use.

MARK DISTRIBUTION

Task	Marks
Task 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3
Task 2 – Basic Regular Expression Patterns	4
Task 3 – Regular Expressions for Tokenisation and Normalisation	4
Task 4 – Finite State Automata: Design, Non-determinism and Equivalence	5
Task 5 – Inflectional Morphology	3
Task 6 – Derivational Morphology	4
Task 7 – Finite-State Morphological Parsing	5
Task 8 – Orthographic Rules and Two-Level Morphology	4
Task 9 – The Porter Stemmer	5
Task 10 – Implement and Test the Pipeline	3
Task 11 – Compare, Recommend and Defend the Normalisation Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**